

Consigna para el Avance de Proyecto Final 3

1. Logro a evaluar:

Al finalizar la unidad, el estudiante en base a una arquitectura definida, desarrolla la solución informática para el proyecto mediante el uso del lenguaje Java y herramientas de apoyo a la codificación.

2. Indicación General:

Durante la semana 12, el avance del proyecto final 3 consistirá en poner en práctica los conocimientos adquiridos para desarrollar la solución informática utilizando el lenguaje Java y aplicando los conceptos de arquitectura MVC, TDD, DAO, SOLID, así como integrando las principales librerías de Java. Además, se debe integrar el control de versiones utilizando Git y GitHub. Recuerda que esta evaluación ofrece flexibilidad.

3. Indicaciones específicas:

Consideraciones formales del informe:

- Desarrollo de la Solución Informática:
 - Construir la arquitectura inicial del proyecto, aplicando los principios de MVC, TDD, DAO y SOLID, y considerando aspectos de seguridad.
 - Utilizar alguna de las librerías de apoyo a la codificación como Google Guava, Apache POI, Apache Commons y Logback para mejorar la eficiencia y funcionalidad del proyecto.
 - Desarrollar un avance del proyecto que cubra al menos el 30% de las funcionalidades requeridas, utilizando las principales librerías de Java identificadas y aplicando buenas prácticas de seguridad.
- Control de Versiones:
 - Integrar el proyecto a un sistema de control de versiones utilizando Git y GitHub.
 - Realizar entregas de avances de desarrollo al 40% del proyecto, asegurando que estén correctamente versionados y documentados en GitHub.
 - Participar en un taller de versionamiento para comprender los conceptos de control de versiones y su aplicación práctica en el proyecto.

Consideraciones para la sustentación oral:

- Preparar una presentación en PowerPoint o equivalente.
- Realizar una exposición oral coherente y fluida, explicando la arquitectura del proyecto, las herramientas utilizadas, los avances de desarrollo y el proceso de control de versiones.
- Demostrar dominio de los conceptos de arquitectura MVC, TDD, DAO, SOLID, control de versiones con Git y GitHub, así como el uso de las principales librerías de Java.
- Todos los participantes deben participar en la exposición.
- La duración máxima es de aproximadamente 10 minutos.

Para el desarrollo del avance del proyecto, deben considerar los siguientes aspectos:

- Diseño de la solución: Se califica que el estudiante diseñe y desarrolle la solución mediante la construcción de la arquitectura inicial del proyecto, aplicando todos los principios de MVC, TDD, DAO y SOLID, y considerando aspectos de seguridad.
- Uso de recursos Java: Se califica que el estudiante utilice los diferentes recursos Java teniendo en cuenta las adecuadas consideraciones de seguridad.
- Uso de control de versiones: Se califica que el estudiante integre en su proyecto el sistema de control de versiones Git y la herramienta de apoyo GitHub.
- Implementación de las interfaces gráficas: Se califica que el estudiante implemente sus interfaces gráficas UX/UI cuyo funcionamiento de la solución cubra el alcance comprometido.
- Construcción del producto final: Se califica que el estudiante construya una solución informática que cumpla con el alcance comprometido, tomando en cuenta los siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).
- Sustentación oral: Se califica la sustentación oral del estudiante, la misma que se presenta de manera organizada (siguen una secuencia lógica), en la que desarrolla ideas con coherencia y fluidez (de manera continua, sin pausas prolongadas). Además, demuestra dominio de la sección que sustenta (Analiza el contexto, Analiza alternativas de solución, Diseña la solución, Diseña prototipo, Construye el producto final), pues las vincula con los conceptos estudiados en el curso.

4. Recomendaciones:

- Designar un responsable de control de versiones que garantice la correcta gestión del repositorio en GitHub.
- Utilizar metodologías ágiles para la gestión del desarrollo del proyecto y mantener una comunicación constante entre los miembros del equipo.
- Consulta la bibliografía recomendada (libros, artículos de revistas, sitios web relevantes, entre otros).
- Evita el plagio. El trabajo debe ser original, evita copiar información de cualquier fuente.
- Realizar pruebas unitarias y de integración utilizando TDD para asegurar la calidad del código desarrollado.
- Solicitar retroalimentación periódica al docente y realizar ajustes en función de las sugerencias recibidas.

5. Criterios de evaluación:

A continuación, podrás encontrar la rúbrica de evaluación con la que será evaluada la actividad. Recuerda que también puedes encontrarla en la plataforma virtual de aprendizaje. Asegúrate de leerla antes de realizar la actividad.

6. Anexos:

El trabajo debe incluir la siguiente información y estructura:

- Diagramas de Arquitectura: Se incluyen diagramas que representan la arquitectura del proyecto, incluyendo la estructura MVC, la implementación de patrones SOLID y la integración de capas DAO.
- Documentación de Código: Se adjunta la documentación técnica generada con herramientas como Javadoc, que describe la estructura del código y el funcionamiento de las principales clases y métodos.
- Referencias bibliográficas consultadas para la elaboración del avance de portafolio final.
- Cualquier otro material relevante que contribuya a la comprensión y evaluación de tu trabajo.

A tomar en cuenta en caso de plagio:

“Todo acto de copiar, intentarlo o dejar copiar, durante una prueba, examen, práctica, trabajo o cualquier asignación académica, usando tanto el medio físico como el electrónico, se encuentra normado en el Reglamento de Estudios y el Reglamento de Disciplina del Estudiante vigentes en el Portal de Transparencia y/o en el Portal del Estudiante”

RÚBRICA DE AVANCE DE PROYECTO FINAL 3

Criterio	Definición de criterio	Estándar Esperado	En Proceso 2	En Proceso 1	Inicial
Diseño de la solución	Se califica que el estudiante utilice los diferentes recursos Java teniendo en cuenta las adecuadas consideraciones de seguridad.	Se califica que el estudiante diseñe y desarrolle la solución mediante la construcción de la arquitectura inicial del proyecto, aplicando al menos 3 de los principios de MVC, TDD, DAO y SOLID, y considerando aspectos de seguridad.	Se califica que el estudiante diseñe y desarrolle la solución mediante la construcción de la arquitectura inicial del proyecto, aplicando al menos 2 de los principios de MVC, TDD, DAO y SOLID, y considerando aspectos de seguridad.	"Se califica que el estudiante diseñe y desarrolle la solución mediante la construcción de la arquitectura inicial del proyecto, aplicando al menos 1 de los principios de MVC, TDD, DAO y SOLID, y considerando aspectos de seguridad.	Se califica que el estudiante diseñe y desarrolle la solución mediante la construcción de la arquitectura inicial del proyecto, aplicando al menos 1 de los principios de MVC, TDD, DAO y SOLID, y sin tomar en cuenta los aspectos de seguridad.
		3	2.5	1	0.5
Uso de recursos Java	Se califica que el estudiante utilice los diferentes recursos Java teniendo en cuenta las adecuadas consideraciones de seguridad.	El estudiante incorpora en su proyecto todas las librerías de apoyo a la codificación como Google Guava, Apache POI, Apache Commons y Logback para mejorar la eficiencia y funcionalidad del proyecto, teniendo en cuenta las consideraciones básicas de seguridad para el tratamiento de la información.	El estudiante incorpora en su proyecto al menos 3 de las librerías de apoyo a la codificación como Google Guava, Apache POI, Apache Commons y Logback para mejorar la eficiencia y funcionalidad del proyecto, teniendo en cuenta las consideraciones básicas de seguridad para el tratamiento de la información.	El estudiante incorpora en su proyecto al menos 2 de las librerías de apoyo a la codificación como Google Guava, Apache POI, Apache Commons y Logback para mejorar la eficiencia y funcionalidad del proyecto, teniendo en cuenta las consideraciones básicas de seguridad para el tratamiento de la información.	El estudiante incorpora en su proyecto al menos 1 de las librerías de apoyo a la codificación como Google Guava, Apache POI, Apache Commons y Logback para mejorar la eficiencia y funcionalidad del proyecto, sin tomar en cuenta las consideraciones básicas de seguridad para el tratamiento de la información.
		2	1.5	1	0.5

Uso de control de versiones	Se califica que el estudiante integre en su proyecto el sistema de control de versiones Git y la herramienta de apoyo GitHub.	El estudiante incorpora el uso del Git en su proyecto, dejando evidencia del 100% de sus avances y actualizaciones de los entregables del proyecto en la plataforma GitHub.	El estudiante incorpora el uso del Git en su proyecto, dejando evidencia del 70% de sus avances y actualizaciones de los entregables del proyecto en la plataforma GitHub.	El estudiante incorpora el uso del Git en su proyecto, dejando evidencia del 50% de sus avances y actualizaciones de los entregables del proyecto en la plataforma GitHub.	El estudiante incorpora el uso del Git en su proyecto, dejando evidencia del 25% o menos de sus avances y actualizaciones de los entregables del proyecto en la plataforma GitHub.
		3	2	1	0.5
Implementación de las interfaces gráficas	Se califica que el estudiante implemente sus interfaces gráficas UX/UI cuyo funcionamiento de la solución cubra el alcance comprometido.	El estudiante implementa sus interfaces gráficas UI/UI cuyo funcionamiento de la solución cubre el 100% del alcance comprometido.	El estudiante implementa sus interfaces gráficas UI/UI cuyo funcionamiento de la solución cubre el 70% del alcance comprometido.	El estudiante implementa sus interfaces gráficas UI/UI cuyo funcionamiento de la solución cubre el 50% del alcance comprometido.	El estudiante implementa sus interfaces gráficas UI/UI cuyo funcionamiento de la solución cubre el 25% o menos del alcance comprometido.
		6	4	2	0.5
Construcción del producto final	Se califica que el estudiante construya una solución informática que cumpla con el alcance comprometido, tomando en cuenta los siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).	El estudiante construye una solución informática considerando todos los siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).	El estudiante construye una solución informática considerando solo 3 de siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).	El estudiante construye una solución informática considerando solo 2 de siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).	El estudiante construye una solución informática considerando sólo uno de los siguientes criterios: 1) completa (cubre el alcance comprometido), 2) coherente (la documentación y el código están alineados), 3) buenas prácticas (usa librerías adecuadas, patrones de diseño, software de control de versiones, entre otros) y 4) autoría (el código fue hecho por el estudiante o lo domina).
		4	3.5	2	0.5

Sustentación oral	Se califica la sustentación oral del estudiante, la misma que se presenta de manera organizada (siguen una secuencia lógica), en la que desarrolla ideas con coherencia y fluidez (de manera continua, sin pausas prolongadas). Además, demuestra dominio de la sección que sustenta (Analiza el contexto, Analiza alternativas de solución, Diseña la solución, Diseña prototipo, Construye el producto final), pues las vincula con los conceptos estudiados en el curso.	Las ideas siguen una secuencia lógica acorde a la estructura de presentación establecida. Además, las ideas se desarrollan de manera continua, sin pausas (fluidez) y son coherentes con el desarrollo del tema. Asimismo, demuestra dominio de la sección que sustenta, pues vincula sus ideas con los conceptos abordados en el curso.	Las ideas siguen una secuencia lógica acorde a la estructura de presentación establecida. Además, las ideas se desarrollan de manera continua, sin pausas (fluidez) y son coherentes con el desarrollo del tema. Sin embargo, no demuestra dominio de la sección que sustenta, pues no vincula sus ideas con los conceptos abordados en el curso.	Las ideas siguen una secuencia lógica acorde a la estructura de presentación establecida. Además, las ideas se desarrollan de manera continua, sin pausas (fluidez) pero son poco coherentes con el desarrollo del tema (hay ciertos vacíos de información y por momentos se desvía del tema). Asimismo, no demuestra dominio de la sección que sustenta, pues no vincula sus ideas con los conceptos abordados en el curso.	Las ideas siguen una secuencia lógica acorde a la estructura de presentación establecida. Sin embargo, las ideas se desarrollan de manera discontinua, hay pausas o interrupciones. Las ideas son poco coherentes con el desarrollo del tema (hay ciertos vacíos de información y por momentos se desvía del tema). Asimismo, no demuestra dominio de la sección que sustenta, pues no vincula sus ideas con los conceptos abordados en el curso.
		2	1.5	1	0.5